Structure, Exact Algorithms and Information Limits in a Unit-Latency Reorder-Buffer Model

Research draft, 14 October 2026 (Asia/Singapore)

ILP1-20261014. Correctness and historical novelty are separate questions.
No author list, publication license, submission, or DOI is determined by this draft.

Abstract

We study offline and online unit-job scheduling with eager in-order admission, bounded unretired occupancy, and in-order retirement. We prove an exact execution-time reduction to precedence scheduling by adding every edge from index i to index j with $i + B \leq j$. This is a specialized formal bridge to established resource-event graph methods, not a claim to have invented them. We give exact algorithms and independently checked certificates, and exhibit an exact polynomial optimizer for arbitrary issue width W when the window has capacity $B = W + 1$. Perfect saturation then has a linear structural test. More generally, a horizon with $S = HW - N$ unused issue slots confines every feasible trace to a strip of width $2(B - W) + S$, yielding fixed-parameter feasibility in $(B - W) + S$. We also exhibit two-input programs whose minimum sufficient window falls from N to 3 when issue width increases from 2 to 3. This unbounded threshold change costs only one extra cycle at window two, highlighting the distinction between exact and approximate saturation. A separate chain-prefix family has linear window requirements even at fixed positive relative tolerance. Under a precise hidden-future information model, a universally optimal online policy exists exactly when $W = 1$ or $W \geq B$. At $B = 3, W = 2$, a simple randomized policy has the sharp strict ratio $5/4$, proved by a joint potential invariant. A two-input repeated construction preserves this lower bound with an additive constant. Lean 4.34.1 verifies the operational reduction, existence of an optimum, an executable exact enumeration algorithm, resource caps, Bellman certificates, the actual-trace strip bound, and a four-job arbitrary-policy online obstruction. General resource families and online classifications have independent written review but are not yet fully formalized. Historical originality remains unconfirmed.

1 Introduction

For a fixed program, increasing processor resources cannot worsen its optimal duration. This does not imply that the least amount of one resource needed to attain an optimum changes monotonically with another resource. Nor does the existence of an offline optimum imply that a finite-information scheduler can always select it. We separate these questions in a deliberately narrow model.

The main structural examples concern exact resource thresholds, rather than real hardware speedups. A separate algorithmic result exploits how completed but unretired jobs remove scheduling choices, yielding polynomial exact optimization for a window one entry wider than issue width. Another result classifies the resource regimes in which hidden future dependencies obstruct universal online optimality. The foundational reduction and algorithms are included to make these claims precise, reproducible, and amenable to formal checking. The result catalogue and complete audit records accompany this draft in the research workstream.

2 Formal model

The program consists of N fixed operations with indices $0, \ldots, N-1$. Its true dependency graph E has only forward edges $i < j$. Operations are deterministic, have unit latency, and execute exactly once. Two-input, one-output operations suffice for the principal examples. SSA or versioned registers remove naming dependencies; this assumes pure operations and preserves producer values. Instruction elimination and algebraic rewriting are excluded; an isolated operation may also produce an observable output.

At a cycle boundary let D be the set of completed operations and let r be the least incomplete index, or N at termination. With $B, W \geq 1$, the eager window is $[r, \min(N, r + B))$. Completed entries behind r remain in the window until earlier operations finish. In one cycle any subset of at most W ready, incomplete window entries may issue. A predecessor must have completed in a previous cycle. Issued jobs complete at the cycle end; the whole completed prefix retires and the next cycle refills immediately. There is no same-cycle dependent bypass.

A schedule is a map $t : \{0, \ldots, N-1\} \to \mathbb{N}$ of issue times. For nonempty programs its duration is $1 + \max_j t(j)$; empty programs take zero cycles and have no defined IPC. Let $T^*(E, B, W)$ be the minimum duration and $IPC^* = N/T^*$. Both resources may be capped at N for nonempty inputs; effective width is at most B.

An offline planner sees the entire graph. An online planner sees only admitted operations, their incoming true dependencies, and execution history. It may know N, but receives no oracle for future uses of visible values. All online lower bounds below use this observation model.

3 Related work and claim boundaries

Wall's report studies instruction-level parallelism under idealization [1]; Dubey, Adams, and Flynn analyze instruction windows [2]. The latter full text was not obtained in this audit. Tullsen and Calder already add window-edge dependencies to model ordered window admission in a 1998 report [4]. Fields, Rubin, and Bodík use execution-event DAGs with resource edges [3]; later work explicitly records FIFO/ROB commit-to-dispatch constraints [5]. Our all-prefix execution-only reduction must be read as a specialization of this prior modeling tradition.

The all-prefix constraints also occur as discrepancy precedences in Cire–van Hoeve's sequencing MDD formulation, whose section9 gives an exponential-in-discrepancy width bound [7]. Thus neither these constraints nor a general finite-frontier state representation is new. Our current algorithm questions instead concern variable W, the difference $B - W$, and a total idle-slot budget. Recent work parameterizes scheduling by time-window overlap or incomparable-graph degeneracy; the workstream audit records these sources and access limits. Those parameters can grow with W even when the parameters here are fixed, so their inspected bounds do not directly imply the results below. This distinction is not novelty clearance.

Bellman recursion, finite exhaustive search, precedence longest paths and Graham's list-scheduling guarantee are classical. At width two the reduced problem is covered by Coffman–Graham's polynomial algorithm [6, 8]. We do not claim a new solution of unrestricted precedence scheduling. A September 2026 OpenAI artifact claims a polynomial three-machine result with Lean sources [9]; its actual endpoint was inspected but its full dependency closure was not independently compiled here. Neither unconditional acceptance nor a statement that this question simply remains open is justified by our current audit.

Hidden-successor adversaries and advice-counting methods also have prior art; the workstream literature notes compare different release and observation models. No exact equivalent of all the threshold formulas below was located, but the search is incomplete. All prospective novelty is unconfirmed.

4 Static reduction and exact algorithms

Theorem 1 (Exact window representation)**.** *Define*

$$E_B = E \cup \{(i, j) : i + B \le j < N\}.$$

A time assignment is legal in the ROB model if and only if it strictly respects E_B and every time layer has at most W jobs.

Proof. When j issues, every index $i \le j - B$ must already have retired, hence completed in a previous cycle. This gives all the artificial precedences; true data and issue-width conditions are necessary as well.

Conversely replay the time layers. Before $t(j)$ all indices $i \le j - B$ have completed. Thus the retirement frontier satisfies $r > j - B$ when $j \ge B$, and admission is immediate when $j < B$. Since j is still incomplete, $r \le j < r + B$. It is in the window, all true predecessors are ready, and the proposed layer fits the issue width. Induction over time proves legality, including schedules with idle cycles. □

Only adding the offset edge $(j - B, j)$ is insufficient: for $N = 4, B = 2$ and $E = \varnothing$, the times $(1, 0, 2, 1)$ satisfy both offset edges but violate the required edge $0 \to 3$.

Proposition 2 (Exact search and certificates)**.** *An optimal finite schedule exists. If $A(D)$ is the ready window set, its value satisfies*

$$V(D) = 1 + \min_{\varnothing \ne S \subseteq A(D),\ |S| \le W} V(D \cup S), \qquad V(\{0, \dots, N-1\}) = 0.$$

A legal K-cycle witness plus a potential v on every reachable state with $v(\varnothing) = K$, $v(\text{all}) = 0$ and $v(D) \le 1 + v(D \cup S)$ for every legal transition proves optimality.

Proof. The earliest incomplete operation is ready because all its true predecessors have smaller indices. Issuing it repeatedly finishes within N cycles. Idle steps are unnecessary, and each nonempty step strictly increases $|D|$. Induction on $N - |D|$ proves the recurrence and constructs its attaining schedule. Telescoping the potential inequality along any execution gives the lower bound K; the legal witness gives the opposite bound. □

The independent checker enumerates successors itself and rejects omitted reachable states. This is a full-state certificate, not a compact one. There are at most $1 + N2^{B-1}$ completed-set states. Labeling each frontier position completed, selected, or unselected bounds aggregate transitions by $O(N3^B)$ before polynomial feasibility-checking factors.

For $W \ge B$, any E_B antichain lies in fewer than B index gaps and has at most B vertices. Earliest precedence times are therefore feasible and $T^*(E, B, W) = h(E_B)$, where height counts vertices. The recurrence

$$e(j) = 1 + \max\left(\max_{(i,j) \in E} e(i),\ \max_{i \le j - B} e(i) \right),$$

with empty maximum -1, is evaluated in $O(N + |E|)$ using prefix maxima. For $W \ge N$, binary search on B computes B_{sat} in $O((N + |E|) \log N)$. A height-optimal schedule, a true critical path, and a longer augmented path at $B - 1$ (only when $B > 1$) form an $O(N)$ certificate of its minimality. At $B = 1$ no smaller positive window needs exclusion.

For any fixed true-dependency-respecting schedule, define

$$\beta(t) = \max\big(\{1\} \cup \{j - i + 1 : i < j,\ t(i) \ge t(j)\}\big), \quad \omega(t) = \max_c |t^{-1}(c)|.$$

The reduction gives legality exactly when $B \ge \beta(t)$ and $W \ge \omega(t)$. Minimizing these footprints over the appropriate optimal schedules characterizes saturation, but is not itself an efficient general algorithm.

5 Polynomial optimization with one extra window entry

Call an action non-delay if it issues $\min(W, |R(D)|)$ jobs, where $R(D)$ is the ready set. This specifies a batch size, not a priority order.

Proposition 3 (Completion dominance and non-delay optimality)**.** *If two reachable states have completed sets $D \subseteq D'$, every continuation from D can be simulated from D' in no more cycles. Consequently every reachable state has a non-delay optimal continuation.*

Proof. Replay the original actions, deleting jobs already complete in the simulating state. True predecessors stay complete. Retirement advances from r to $r' \geq r$; a retained job satisfies $j \geq r'$ because it is incomplete, and $j < r + B \leq r' + B$ because it was originally admitted. Thus it remains admitted and ready. Deletion cannot exceed issue width. This preserves completed-set inclusion at each boundary and finishes no later. Enlarge an optimal first action to the non-delay size and apply this simulation. Induction on the number of unfinished jobs gives a non-delay optimum at every later state. □

Write $h(D) = |D| - r(D)$ for completed but unretired jobs. They still occupy window entries, so $|R(D)| \leq B - h(D)$. If $B = W + 1$ and $h \geq 1$, the non-delay action is therefore uniquely determined: issue every ready job. Choices occur only at clear-prefix states $D = [0, i)$, and only when all $W + 1$ window entries are ready.

Theorem 4 (Exact prefix dynamic program)**.** *For arbitrary finite forward dependencies and arbitrary input $W \geq 1$, the optimum at $B = W + 1$ and an attaining schedule can be computed in*

$$O(N \min(B, N)(N + |E|)) \text{ time}, \qquad O(N + |E|) \text{ space}$$

using indexed-graph operations. No divisibility assumption on N is needed.

Proof. At prefix i, enumerate its non-delay first actions: one if at most W jobs are ready, otherwise the B choices of an omitted ready job. After each action, simulate the unique non-delay actions until the first new clear prefix j. Every nonterminal cycle issues at least the oldest job when its action is unique; the first action is also nonempty. Hence this macro ends in at most $N - i$ cycles, has length ℓ, and $j > i$. The non-delay optimum must follow one such macro. Conversely every macro is a legal continuation. Thus

$$F_N = 0, \qquad F_i = \min_{\text{first actions } A} \big(\ell(i, A) + F_{j(i,A)}\big).$$

Backward induction gives the exact remaining optimum at every prefix.

There are at most $N \min(B, N)$ macros. For each, initialize predecessor counters from its starting prefix in $O(N + |E|)$ time. Monotone admission and retirement pointers admit and complete each job once during that macro; each outgoing edge decrements a counter once. Newly ready jobs wait until the next cycle. Thus a macro costs $O(N + |E|)$. Store only its winning first omission, cost and endpoint at each prefix, then replay winning macros for the schedule. This gives the bounds. Empty programs return zero, and arbitrarily large W does not cause iteration over absent jobs. □

This is an offline algorithm: evaluating choices uses dependencies that may be outside the current execution window. A forced macro can have length linear in N even at fixed $B = 4, W = 3$, so a constant macro-length assumption would be invalid.

For general normalized excess $c = B - W \geq 1$, branching requires $h < c$. There are at most $N \sum_{h=0}^{c-1} \binom{B-1}{h}$ branching states. The total number of outgoing macros is at most

$$N \sum_{h=0}^{c-1} \binom{B-1}{h} \binom{B-h}{W} \leq N(2^c - 1) \binom{B}{c},$$

by replacing $\binom{B-1}{h}$ with $\binom{B}{h}$ and applying the identity $\binom{B}{h}\binom{B-h}{c-h} = \binom{B}{c}\binom{c}{h}$. Compress each unique-action segment, including an initial forced segment if necessary. Completed-set size strictly increases along every macro, giving an exact acyclic state graph. This is an XP algorithm in c, not a fixed-parameter algorithm in c alone. At $c = 0$ every action is unique.

5.1 A linear perfect-saturation criterion

Suppose $W \geq 2$, $B = W + 1$ and $N = kW$. Partition the jobs into k consecutive blocks of W entries. A singleton macro executes one block in one cycle. A pair macro swaps the last job of its left block with the first job of its right block, then executes the resulting sets in two successive cycles. A macro is legal when every internal true edge points strictly from an earlier to a later macro cycle.

Theorem 5 (Exact-work-bound tiling)**.** *$T^* = k$ if and only if these blocks can be tiled by legal singleton and pair macros. Every k-cycle schedule has this decomposition. The decision and a positive or negative certificate take $O(N + |E|)$ graph operations.*

Proof. Every cycle must issue W jobs. At a clear-prefix state with at least two blocks remaining, a full action omits exactly one of $W + 1$ entries, at one-based position r. Only the first $r - 1$ entries retire, leaving at most r unfinished window entries next cycle. Full width forces $r \geq W$. For $r = W + 1$ this is a singleton macro. For $r = W$ the next full action is forced and gives exactly the pair macro. Both end at a clear prefix. The final single block, if present, is a singleton. Conversely legal macros satisfy admission and return to clear prefixes, so a tiling gives k cycles.

Make a path on blocks, keeping an edge iff its pair macro is legal and marking a vertex iff its singleton is legal. An even component can be paired. An odd component can be covered iff it has a marked odd local position: use that singleton and pair both sides; conversely the leftmost singleton of any tiling must have odd position. Each true edge invalidates at most one singleton and two adjacent-pair macros. One graph scan and one path scan therefore suffice. For rejection, supply an odd interval isolated by illegal boundary pair macros and with all odd-position singletons illegal. One actual true edge witnesses each illegality; internal pair flags need not be checked. No full-state certificate is needed. □

6 A sharp completion corridor near the work bound

Let $1 \leq W \leq B$, choose a horizon H with $WH \geq N$, and put $c = B - W$, $S = WH - N$. A trace finishing earlier is padded with terminal idle cycles, so its total unused issue capacity is exactly S.

Theorem 6 (Actual-trace corridor)**.** *At every boundary $0 \leq t \leq H$, every such execution satisfies*

$$[0, \max(0, tW - c - S)) \subseteq D_t \subseteq [0, \min(N, tW + c)), \qquad tW - S \leq |D_t| \leq tW.$$

Proof. For $t < H$, let r be the first incomplete index at t. All jobs completed by $t + 1$ lie below $r + B$: earlier admissions had no larger frontier, and the next action uses the current window. At most $(H - t - 1)W$ jobs fit in the remaining cycles, hence $N \leq r + B + (H - t - 1)W$ and $r \geq tW - c - S$. At $t = H$, use $r = N$ directly. All indices below r are complete.

A job completed in cycle $s \leq t$ was admitted below $r_{s-1} + B \leq (s-1)W + B = sW + c \leq tW + c$. Finally $|D_t| \leq tW$ and $N - |D_t| \leq (H - t)W$ give the count interval. The argument allows idle cycles, truncated windows and empty programs. □

This theorem, including counts derived from actual cycle/lane injections, is verified in Lean for actual ROB traces. The following combinatorial count and algorithm currently have reviewed written proofs.

Theorem 7 (Sharp candidate count and parameterized feasibility)**.** *Each layer has at most*

$$M(c,S)=\sum_{j=c}^{c+S}\binom{2c+S}{j}\le 2^{2c+S}$$

candidate completed sets. This bound is sharp. Exact horizon feasibility and an attaining schedule can be computed in $2^{O(c+S)}\,\mathrm{poly}(N+|E|)$ *time.*

Proof. Embed the variable strip into the untruncated interval $[tW-c-S, tW+c)$. Treat negative indices as fixed complete, and indices at least N as fixed incomplete. If $d=tW-|D_t|$, its selected subset has size $c+S-d$, where $0\le d\le S$. This is an injection into the families counted by M; boundary truncation only removes candidates.

For sharpness choose $W\ge\max(1,c+S)$, $H=2$, $N=2W-S$ and independent jobs. Let $L=W-c-S$, $U=W+c$. For every subset $A\subseteq[L,U)$ of size between c and $c+S$, issue $D=[0,L)\cup A$ in cycle one. It is initially admitted and has size between $W-S$ and W. Its retired prefix is at least L, so its next frontier is at least $L+B=N$. At most W jobs remain, all independent, and finish in cycle two. All M distinct middle states therefore occur on one input. This is not a computational lower bound; the input itself is easy.

Generate the layer candidates and test every consecutive-layer pair for the actual ROB transition: containment of completed sets, at most W new jobs, admission using the actual first missing index, and all true parents already complete. Empty additions are allowed. Every edge is sound; the corridor puts every feasible trace inside the graph. Reachability and predecessors decide feasibility and recover a schedule. There are at most M^2 pairs per layer; graph/mask arithmetic is polynomial. If $H\ge N$, use the sequential witness instead, so a binary-encoded horizon is harmless. No step enumerates all 2^B ready subsets. □

At $c=1, S=0$ there are only two candidate forms: a clear prefix or its boundary swap. A swapped state cannot be followed by another swapped state at full width because the next required final index is outside admission. This recovers the preceding tiling result. For $S>0$, appending independent dummy jobs is not a reduction to $S=0$: at $B=4, W=3$, five jobs with sole true edge $0\to 1$ finish in two cycles, but adding a sixth independent job makes two cycles impossible.

7 Unbounded changes in exact resource thresholds

For nonempty programs define

$$B_{\mathrm{sat}}(E,W)=\min\{B: T^*(E,B,W)=T^*(E,N,W)\},$$

$$W_{\mathrm{sat}}(E,B)=\min\{W: T^*(E,B,W)=T^*(E,B,B)\}.$$

Optimal duration is nonincreasing in each resource, since more window space removes artificial edges and more issue capacity relaxes layer size. The threshold functions do not inherit that monotonicity.

Theorem 8 (A fixed-width window collapse)**.** *For every integer $K\ge 2$ there is a program P_K of $N=2K$ operations, with true indegree and outdegree at most two, such that*

$$B_{\mathrm{sat}}(P_K,2)=2K,\qquad B_{\mathrm{sat}}(P_K,3)=3.$$

Both unrestricted-window optima equal K.

Proof. Job 0 is isolated. The other jobs are pair layers $\{1,2\},\{3,4\},\ldots,\{2K-3,2K-2\}$ followed by singleton $\{2K-1\}$. Between successive layers add all forward edges. Every nonisolated job lies on a K-vertex path. Consequently any K-cycle schedule forces the job in layer ℓ into cycle $\ell-1$.

At width two every pair layer fills its cycle; job 0 must share the final cycle with job $2K-1$. This is legal at $B=N$ and meets the height bound. At every $B<N$, however, the artificial edge $0\to 2K-1$ prohibits that common time. Hence $B_{\mathrm{sat}}(P_K,2)=N$.

At $B=W=3$, issue $\{0,1,2\}$ first, then successive pairs, then the final singleton. Every batch is admitted and the duration is K. At $B\le 2$, job 2 is not admitted initially, contradicting its forced initial time in every K-cycle schedule. Thus the second threshold is exactly three. □

The additive threshold drop $2K-3$ and the ratio $2K/3$ are unbounded, even for fixed issue widths. This does *not* imply an unbounded performance gain. At width two and every $2\le B<2K$, the exact duration is only $K+1$: issue $\{0,1\}$, then $\{2\}$, then the remaining pair layers and the final singleton. Window two already suffices for this schedule, and the theorem excludes K cycles. At window one the duration is $2K$.

Define $B_\varepsilon(E,W)$ by allowing duration at most $(1+\varepsilon)T^*(E,N,W)$. The exact curves for this family are

	$0\le\varepsilon<1/K$	$1/K\le\varepsilon<1$	$\varepsilon\ge 1$
$W=2$	$2K$	2	1
$W=3$	3	2	1

Thus this example's huge exact threshold is fragile under fixed relative tolerance as K grows. The following separate construction shows that such fragility is not universal.

Theorem 9 (A depth-two width-saturation curve)**.** *Let $m\ge 2$ and $1\le r\le\lfloor m/2\rfloor$. Take r initial isolated roots, then m required roots, then $m-r$ sinks. Partition the required roots into r pairs and $m-2r$ singletons, assigning each group to a different sink. For $1\le B\le 2m$,*

$$T^*(E,B,W)\le 2 \quad\Longleftrightarrow\quad B\ge m+r \ \text{ and } \ W\ge m+\min(r,2m-B).$$

In particular, $W_{\mathrm{sat}}(E,B)=m+\min(r,2m-B)$ for $m+r\le B\le 2m$.

Proof. All m required roots must issue first, so the last one must be initially admitted: $B\ge m+r$. Admission of the final job in the next cycle forces the first $2m-B$ jobs to complete initially. The union of these jobs and the required roots has size $m+\min(r,2m-B)$, giving the width bound. Conversely issue exactly that union first. Its prefix retirement admits the whole remaining program, every sink is ready, and at most m jobs remain for the second cycle. Any true edge excludes a one-cycle schedule. □

As B grows from $2m-r$ to $2m$, the sufficient width decreases by r. Original indegree is at most two and outdegree at most one.

Theorem 10 (Robust linear window demand)**.** *Let Q_K consist of a chain in indices $0,\ldots,K-1$, followed by K independent operations, where $K\ge 1$. For all $B\ge 1, W\ge 2$,*

$$T^*(Q_K,B,W)=K+\left\lceil\frac{\max(0,K-B+1)}{\min(B,W)}\right\rceil.$$

Writing $q=\lfloor\varepsilon K\rfloor$ for $\varepsilon\ge 0$, its exact approximate saturation threshold is

$$B_\varepsilon(Q_K,W)=\max\left(\left\lceil\frac{K+1}{q+1}\right\rceil,\ K+1-qW\right).$$

For every fixed $W\ge 2$ and $0\le\varepsilon<1/W$, this threshold is $\Omega(N)$, although true indegree and outdegree are at most one.

Proof. The final chain operation completes no earlier than the end of the Kth cycle. At the beginning of its issue cycle, the retirement frontier is at most $K-1$. Hence at most $B-1$ independent operations have even been admitted, including any issued alongside this last chain operation. At least $Q = \max(0, K-B+1)$ independent operations remain afterward, requiring $\lceil Q/\min(B,W) \rceil$ further cycles. Delaying the chain cannot evade this admission bound.

For attainment, run the chain without gaps. In each of its last $\min(K, B-1)$ cycles also issue the next independent operation. These operations are admitted as the chain prefix retires, and width at least two suffices. After the chain finishes, the remaining Q independent operations can be issued in consecutive batches of size $\min(B,W)$. This proves the duration formula, including $B=1$ and $B>N$.

At unrestricted window the optimum is K, so the allowed extra cycles are q. The condition for a window B is exactly $K+1 \le B + q\min(B,W)$. The identity

$$B + q\min(B,W) = \min((q+1)B,\ B+qW)$$

turns this into the two lower bounds in the formula. In particular $B_\varepsilon \ge K+1-W\lfloor \varepsilon K \rfloor \ge (1-W\varepsilon)K+1$, proving the final claim. □

For width two and $0 \le \varepsilon < 1$, this simplifies to $\max(2, K+1-2\lfloor \varepsilon K \rfloor)$; at $\varepsilon \ge 1$ it is one. At width one, the unrestricted baseline is instead $2K$, and the relative saturation window is one for every nonnegative tolerance.

Corollary 11 (A robust single-slot trade-off)**.** *Fix an integer $W \ge 2$ and $1/(W+1) \le \varepsilon < 1/W$. On the same family, $B_\varepsilon(Q_K, W) = \Omega(N)$ whereas $B_\varepsilon(Q_K, W+1) \le W+1$. Thus increasing issue width by one can reduce the approximate sufficient window by an unbounded factor at fixed positive tolerance.*

Proof. The lower bound follows from the theorem. Since $q \ge \lfloor K/(W+1) \rfloor$, integer division gives $K+1 \le (W+1)(q+1)$. Both terms in the formula for width $W+1$ are therefore at most $W+1$. □

For width two this applies to every fixed $1/3 \le \varepsilon < 1/2$. This concerns approximate thresholds: the exact threshold of Q_K is $K+1$ for all $W \ge 2$. Program order matters. Interleaving the same chain with its independent operations allows $B = W = 2$ to finish in K cycles, but reordering is excluded here. The relevant object is a DAG with its fixed topological program order, not an unlabeled partial order alone.

8 Information limits for online scheduling

Theorem 12 (Universal-optimality classification)**.** *For fixed positive B, W, a deterministic online policy optimal on every finite R1 program exists if and only if $W = 1$ or $W \ge B$. The same classification holds for randomized policies required to be optimal in expectation on every input.*

Proof. At $W=1$, repeatedly issue the oldest ready operation: every nonidle execution takes N cycles. At $W \ge B$, issue every ready operation. The static reduction identifies this with augmented earliest-time layering, which is optimal and needs no future information.

Suppose $B > W \ge 2$ and put $L = \lceil (B-W)/(W-1) \rceil$. The first B operations are roots. An L-job suffix chain starts from a hidden root $x \in \{1, \dots, B-1\}$. These inputs have identical initial views and the same known length $B+L$.

For every x, issue any W roots containing $0, x$ first. Then issue each successive chain job along with up to $W-1$ smallest remaining roots. Before suffix job k, the completed root prefix has length at least $\min(B, 1+(k-1)(W-1)) \ge k$, so admission is legal. The capacity inequality

$W + L(W-1) \geq B$ finishes all roots. This realizes $L+1$ cycles and meets the true chain lower bound.

Every deterministic first action either omits 0, delaying suffix admission, or includes 0 and omits some possible x. The corresponding fixed input cannot start its suffix in cycle one, and needs at least $L+2$ cycles. For a randomized first action, average over uniformly chosen x. An action containing 0 hits at most $W-1$ of the $B-1$ choices, and one omitting 0 fails every choice. Positive average failure implies some fixed input with expected duration strictly greater than its optimum. □

The lower-bound family uses only a true chain and isolated vertices. It is an information obstruction, even without a computational complexity bound on the policy. On this family, the exact randomized minimax ratio is $1 + (B-W)/((B-1)(L+1))$; choosing 0 plus a uniform $(W-1)$-subset and using one extra root-only cycle after a miss attains it. The complete continuation and advice-counting proofs are in the accompanying theory note.

At $B = 3, W = 2$, the four-job choices $E = \{1 \to 3\}$ and $E = \{2 \to 3\}$ force distinct optimal initial batches $\{0,1\}$ and $\{0,2\}$. Every deterministic policy has ratio at least $3/2$ on one. Graham's $2 - 1/W$ bound applies to every non-delay policy on the augmented DAG, proving the global strict deterministic minimax ratio is exactly $3/2$ at these resources. The next section sharpens the global randomized value to exactly $5/4$. The finite witnesses establish strict ratios with *no additive constant*. A separate repeating construction below also settles the definition allowing a fixed additive constant.

9 The sharp randomized bound at a three-entry window

Fix $B = 3, W = 2$. Policy R always issues the oldest unfinished operation. When all three window entries are unfinished and ready, it chooses one of the two younger operations uniformly, using fresh randomness. Otherwise it issues all ready jobs. The following bound applies to every fixed program; it does not give the adversary access to realized random choices.

Theorem 13 (Sharp randomized guarantee)**.** *Let F be the offline optimum, and E the expected duration of R. Then*

$$4E \leq N + 3F \leq 5F.$$

For nonempty inputs the global strict randomized online minimax ratio at these resources is exactly $5/4$ against an oblivious adversary.

We give the complete upper-bound argument. The preceding four-job pair supplies the lower bound: under the uniform distribution over its two fixed programs, any initial action succeeds in two cycles with probability at most $1/2$. Thus some fixed input has expected duration at least $5/2$ and optimum two.

First, when $B = W + 1$, some optimal non-delay schedule includes the oldest unfinished job every cycle. Choose the lexicographically least time vector among all optimal schedules. It is non-delay because a ready job can move into any unused issue slot. If oldest a is omitted, all W other window entries must issue; retirement stalls and the next cycle issues only a. Swap a with any job of the first batch. The two-cycle endpoint is unchanged and the time vector decreases, a contradiction. The swapped schedule need not itself be non-delay: the minimum was over all optimal schedules.

For a completed-prefix state beginning at index i, let F_i, E_i be its optimal and expected remaining times, and $n_i = \max(0, N-i)$. Beyond the program all three quantities are zero. Write $a = i, b = i+1, c = i+2, d = i+3$, omitting nonexistent jobs. If just a is ready, advance one job in one cycle. If only younger job b is ready, advance two jobs in one cycle. Issuing a, c forces a second cycle with b and, if ready, d. This advances $\ell = 4$ jobs if d exists and $b \to d$ is absent, otherwise $\ell = 3$. When both younger jobs are ready, optimality takes the minimum of

the two macro-actions $(1, i+2)$ and $(2, i+\ell)$; R takes their average. The normalization just proved establishes completeness of this recurrence.

We also have $0 \le F_i - F_{i+1} \le 1$: completing an additional prefix job cannot harm a continuation, while issuing oldest alone costs one cycle. Define

$$K_i = 3F_i + n_i - 4E_i.$$

We prove jointly, by descending induction, that $K_i \ge 0$, and that

$$i < N, \quad F_i = F_{i+1} \quad \Longrightarrow \quad K_i + K_{i+1} \ge 1. \tag{*}$$

The restriction $i < N$ excludes two empty suffixes. At the final job, $F_i = E_i = n_i = 1$, hence $K_i = 0$ and the equality antecedent is false.

A forced macro-action advancing ℓ jobs in z cycles gives $K_i = \ell - z + K_{i+\ell}$. The increments for $(\ell, z) = (1,1), (2,1), (3,2), (4,2)$ are $0, 1, 1, 2$. In a randomized state put $p = i+2, q = i+\ell, x = F_p - F_q$. Direct substitution gives

ℓ	x	K_i
3	0	$(K_p + K_q)/2 - 1/2$
3	1	$(K_p + K_q)/2 + 1$
4	0 or 2	$(K_p + K_q)/2$
4	1	$(K_p + K_q)/2 + 3/2$

These exhaust the possibilities because each consecutive F difference is zero or one. Every row is nonnegative by the tail induction hypothesis, except possibly $\ell = 3, x = 0$. There p is nonempty, $q = p + 1$ and $F_p = F_q$, so tail invariant (*) cancels the negative half. This proves $K_i \ge 0$ before proving (*) at the same index.

For (*), a forced singleton would give $F_i = 1 + F_{i+1}$, excluding its antecedent. All other forced cases and the randomized cases $x = 1$ give $K_i \ge 1$. It remains to handle three randomized cases, with a, b, c all present and ready, hence with no true edges among them.

If $\ell = 3, x = 0$, then $F_c = F_d$. Job d must exist, since c is nonempty and has positive value. The edge $b \to d$ therefore exists. At the suffix beginning at b, only younger job c is ready, so $K_b = 1 + K_d \ge 1$. Together with $K_a \ge 0$ this proves (*).

If $\ell = 4, x = 2$, put $e = i + 4$. Then $F_c = F_d + 1 = F_e + 2$. Without the edge $c \to d$, the legal first pair c, d would give $F_c \le 1 + F_e$, a contradiction. Thus $c \to d$ exists, and at suffix b only younger job c is ready. Again $K_b = 1 + K_d \ge 1$.

Finally suppose $\ell = 4, x = 0$, so $F_c = F_e$. If $c \to d$ exists the preceding argument applies. Otherwise the pair c, d is legal but costs $1 + F_e > F_c$. The recurrence forces the alternative c, e to exist, be ready and be optimal. In particular $c \to e$ and $d \to e$ are absent. Its two-cycle target is $u \in \{e+1, e+2\}$ with $2 + F_u = F_e$. The one-step difference bound excludes $u = e + 1$, so

$$F_e - F_{e+1} = F_{e+1} - F_{e+2} = 1, \qquad F_d = F_e.$$

At suffix b, all b, c, d are ready. Its second macro-action b, d also issues e next cycle because $c \to e$ is absent. Its targets are $d, e+1$, whose optimal values differ by one. This is the good $\ell = 4, x = 1$ row, so $K_b \ge 3/2$. This completes (*) and the induction.

At the initial state, $K_0 \ge 0$ gives $4E \le N + 3F$. Width two gives $N \le 2F$, finishing the upper bound. On each member of the four-job pair, R has $E = 5/2, F = 2$, attaining equality. Equivalently, R achieves in expectation at least three quarters of the optimum number of double-issue cycles. This pairing interpretation does not by itself establish an equivalence to an arbitrary online matching model.

The general statement above has written proof and independent review. Lean verifies its conditional joint-induction kernel and each local min/mean potential identity. The remaining interface contains nine local macro classifications and their structural successor equations; these

are not assumed nonnegativity conclusions. The connection from all ROB programs to this interface, and the probability/scale semantics, are not yet formalized. A prior exhaustive search of 2,796,203 effective local-edge graphs through $N = 12$ found no counterexample; it is supporting evidence, not the proof. Historical novelty of the exact bound remains unconfirmed.

10 Repeating the obstruction without separator cycles

Theorem 14 (Lower bounds with additive constants)**.** *At $B = 3, W = 2$, the best randomized competitive factor against oblivious inputs is $5/4$, even in the definition $\mathbb{E}[T(P)] \le cT^*(P) + \alpha$ with a fixed finite additive constant α. The deterministic factor in the same definition is $3/2$. The lower-bound inputs have true indegree at most two.*

Proof. For $q \ge 1$, concatenate blocks $(a_k, b_k, c_k, d_k) = (4k, 4k+1, 4k+2, 4k+3)$, $0 \le k < q$. Within each block d_k depends on a hidden choice $x_k \in \{b_k, c_k\}$. For $k > 0$, each of a_k, b_k, c_k has exactly the two true predecessors c_{k-1}, d_{k-1}. All indices remain forward and true indegrees are at most two.

The first job of a new block has artificial predecessors a_{k-1}, b_{k-1} and true predecessors c_{k-1}, d_{k-1}. For the next job, window edges already cover the first three previous jobs and its true edge covers the fourth; for the third job, window edges cover all four. Thus no job of the new block can issue before all jobs of the previous block complete. This is strict cycle separation without a separator job or extra cycle. Every block needs at least two cycles by width, and the batches $\{a_k, x_k\}$ followed by the other root and d_k attain two. Consequently the offline optimum is $2q$.

Choose the hidden bits independently and uniformly before execution. Before the first block action, d_k is unadmitted because a_k has not retired. Earlier visible operands reveal no information about x_k; future roots refer only to the fixed identities c_k, d_k. Conditional on every previous observation and all policy random state, the current bit is therefore still fair. Completion in two cycles forces one of the two distinct first actions $\{a_k, b_k\}$ or $\{a_k, c_k\}$, according to the hidden bit. Success probability is at most one half; every failure, including idle or underfilled first choices, needs at least three cycles. Separation and linearity of expectation give average duration at least $5q/2$. For every q there is thus a fixed input with expected duration at least $5q/2$ and optimum $2q$. The inequality $5q/2 \le 2cq + \alpha$ for every q forces $c \ge 5/4$.

For a deterministic policy, select each still-hidden bit to disagree with its first action. This recursively determines one fixed complete program, since the action cannot yet depend on that bit. Every block needs at least three cycles, proving $3q \le 2cq + \alpha$ and $c \ge 3/2$. This argument is only for deterministic policies; the randomized proof used the oblivious product distribution. The preceding strict upper bounds complete both equalities. □

The product-distribution and direct-sum methods are classical. Historical priority of the exact two-input ROB composition is unconfirmed. The general composition has two independent written reviews. Lean verifies its true indegree bound, arbitrary-block separation, exact offline value $2q$, and hidden-job admission barrier in the actual ROB model, including $q = 0$. Conditioning on observation histories and the probabilistic lower bound are still written arguments.

11 Larger windows and a failed policy extension

Proposition 15 (A hidden-subset family)**.** *Let $W \ge 2$, $1 \le m \le W - 1$, $W < B \le 2W - m$, and let $d \ge 1$ satisfy $W - m \le dm$. There is a family of $M = \binom{B-m}{W-m}$ programs with $N = B + m$, true indegree at most d, outdegree at most one and depth at most two, on which the exact randomized minimax ratio is $3/2 - 1/(2M)$. Exactly $\lceil \log_2 M \rceil$ bits of fixed-length pre-decision advice are necessary and sufficient for a deterministic policy always to be optimal within this promised family.*

Proof. The first B jobs are roots. Put $F = \{0, \ldots, m-1\}$ and $U = \{m, \ldots, B-1\}$. Choose a hidden subset $A \subset U$ of size $W - m$ and partition it among the m suffix jobs, each with at most d parents. Empty parts are allowed. All initial observations agree. Any two-cycle completion must issue F first to admit the last job, and A first to satisfy every suffix dependency. Thus its first action is uniquely $F \cup A$, filling width W. Conversely the remaining $B - W$ roots and m suffix jobs fit cycle two. The optimum is two.

The M distinct required actions give success probability at most $1/M$ under uniform hidden input, and therefore expected duration at least $3 - 1/M$ on some fixed input. Guessing A' uniformly attains this value: issue $F \cup A'$, then every remaining root and ready suffix job, which fit because $B - W + m \leq W$. On a wrong guess the blocked sinks all fit cycle three. Advice needs M messages to distinguish the first actions, and encoding A suffices. This is the classical distinguishability counting argument applied to this family. □

The attaining policy is promised the family; this is a global-model lower bound, not a global upper bound. At $B = W+1$, $M = W+1-m$. Taking $W = 2q, m = q, d = 1$ gives a disjoint-edge true graph and lower bound $3/2 - 1/(2(q+1))$, tending to $3/2$ as the resources grow. This is distinct from an asymptotic input-length claim at fixed resources.

At $B = 4, W = 3$, uniform omission of one younger job when all four are ready does not guarantee $4/3$. For $N = 9$ with zero-based true edges

$$\{1 \to 4,\ 2 \to 4,\ 2 \to 5,\ 5 \to 8\},$$

the optimum is three, attained by $\{0,1,2\}; \{3,4,5\}; \{6,7,8\}$. The uniform policy has durations three, four and five with probabilities $1/9, 2/3, 2/9$, respectively. Its expected ratio is $37/27 > 4/3$. An independent constraint solver and direct probability-tree replay verify these exact values. Exhaustive effective local-edge search through $N = 8$ found no smaller counterexample to this policy. A different online policy might still attain $4/3$; that minimax question remains open here.

12 Computational and formal verification

The operational Python solver uses completed-set memoization. A separately implemented time-assignment constraint solver shares neither that recurrence nor its state encoding. Operational replay and Bellman checking are also separate implementations. Evidence preserves seeds, source hashes, timings, and exact tested ranges. The following datasets overlap and must not be added as if they were disjoint experiments.

Audit	Actual scope
Root exhaustive baseline	All 1,100 forward DAGs with $N \leq 5$; 26,706 resource cases, plus 100 seeded fan-in-two random graphs.
Independent solver agreement	17,378 instances: all 684 fan-in-two DAGs with $N \leq 5$ at all tested resources, plus 1,000 seeded larger instances.
Static/operational bridge	264,121 finite time assignments or replay cases on all 75 DAGs with $1 \leq N \leq 4$.
Fixed-width threshold family	216 horizon-K decisions, $2 \leq K \leq 10$, all windows, widths two and three; independent witness replay.
Depth-two threshold family	25 programs, 5,720 horizon-two decisions, $2 \leq m \leq 10$, all allowed r and all windows/widths.
Approximate saturation family	2,020 exact resource comparisons for $1 \leq K \leq 10$, including oversized windows; 1,185 tolerance thresholds.
Compact certificates	350 independent random instances and 24 deliberately malformed certificates.

Lean 4.34.1 (compiler commit `5045d0056413266e57c625dcd7c365b10e377c52`) uses Std, with no Mathlib dependency. The main modules expose 68 audited declarations; the resource-footprint module adds 18, three supplementary Bellman/search modules add 20, the conditional

randomized-potential module adds 10, and the actual repeated-gadget module adds 20, and work-count/corridor modules add 25 distinct declarations (161 in total). Root recompilation uses fresh temporary compiled objects. Audits permit only Lean's standard `propext`, `Classical.choice` and `Quot.sound`; no new unproved axiom, `sorry`, or `admit` is used. A documented process-path compatibility shim is a host runtime accommodation and does not change the compiler or kernel.

The formal scope includes operational/time-assignment equivalence, finite optimality, resource caps, the exact fixed-schedule minimum-window formula, a concrete executable time-and-lane enumeration algorithm with soundness/completeness/optimality, operational Bellman lower certificates, a general attaining-potential theorem, and the arbitrary-local- policy four-job obstruction. Generic finite-search lemmas and the actual concrete enumerator are distinguished. These proofs do not verify the Python program, the polynomial forced-segment DP, the corridor candidate count or parameterized algorithm, the full general online theorem, or the unbounded saturation families. Finite tests are not universal proofs. Independent agent review is not academic peer review.

For the new polynomial optimizer, independent time-variable constraints and ROB replay agree on 4,472 instances; 26,746 clear-prefix values also agree. A separately implemented general-excess compressed algorithm was checked on 13,380 cases. The corridor implementation has 4,797 independent constraint comparisons, 1,921 feasible witness replays, and 9,384 boundary checks on 2,050 complete legal time maps. The linear saturation theorem has separate graph, abstract macro-pattern and negative-certificate audits. Each saved report gives its actual ranges and seeds; these counts overlap.

13 Limitations and future work

The model omits variable operation latency, functional-unit classes, finite retirement bandwidth, physical-register allocation, memory hierarchy, speculation, branches, exceptions, dynamic power and clock constraints. Its online information model is deliberately explicit and may differ from what a concrete processor can infer. No implementation-level speed or energy claim follows from these theorems.

The next questions are fixed-parameter tractability in $B - W$ alone, efficient optimality certificates for the compressed graph, tight online ratios at other fixed resources, composition of larger hidden-subset families, efficient certificates at finite issue width, and formalization of the general resource and observation theorems. A deep originality audit remains needed, especially for bounded-lookahead and FIFO storage scheduling. All research states and reproducible commands are saved in `STATUS.md` and `HANDOFF.md`; the authorized research branch is `partner/dist-ILP1-20261014`.

References

[1] D. W. Wall. *Limits of Instruction-Level Parallelism.* WRL Research Report 93/6, 1993. https://davidwall.info/papers/WRL-TR-93.6.pdf.

[2] P. K. Dubey, G. B. Adams III, M. J. Flynn. *Instruction Window Size Trade-Offs and Characterization of Program Parallelism.* IEEE Transactions on Computers, 1994. https://doi.org/10.1109/12.278481. Institutional record inspected; full-text access incomplete.

[3] B. Fields, S. Rubin, R. Bodík. *Focusing Processor Policies via Critical-Path Prediction.* ISCA, 2001. https://www.eecg.utoronto.ca/~moshovos/ACA06/readings/critical.pdf.

[4] D. M. Tullsen, B. Calder. *Computing Along the Critical Path.* UCSD Technical Report, October 1998. https://cseweb.ucsd.edu/~calder/papers/UCSD-CSE98-CPProfile.pdf.

[5] S. Navada, N. K. Choudhary, E. Rotenberg. *Criticality-driven Superscalar Design Space Exploration*. PACT, 2010. https://ericrotenberg.wordpress.ncsu.edu/files/2025/06/paper_PACT-19.pdf.

[6] E. G. Coffman Jr., R. L. Graham. *Optimal Scheduling for Two-Processor Systems*. Acta Informatica 1, 200–213, 1972. https://doi.org/10.1007/BF00288685.

[7] A. A. Cire, W.-J. van Hoeve. *Multivalued Decision Diagrams for Sequencing Problems*. Author manuscript revised May2013, section9, Lemma8 and Theorem3. https://www.andrew.cmu.edu/user/vanhoeve/papers/mdd_sequencing.pdf.

[8] R. L. Graham. *Bounds for Certain Multiprocessing Anomalies*. Bell System Technical Journal 45, 1563–1581, 1966. https://doi.org/10.1002/j.1538-7305.1966.tb01709.x.

[9] OpenAI. *Three-machine unit-time precedence scheduling*, family 124, September 2026 artifact. Inspected repository commit `1581d4cbdd69a0faf34b2089614fe2a814316a6a`. https://github.com/openai/math/blob/1581d4cbdd69a0faf34b2089614fe2a814316a6a/lean/docs/124.md.